Resume Analyzer and Job Matcher

CAI4002: Introduction to Artificial Intelligence | Group Project Final Report

Team Members: Martin Dang, Tuan Khang Phan
July 21, 2026

Abstract

This project builds an AI system that measures how well a resume fits a specific job posting and recommends the edits that would raise that fit the most. The completed pipeline extracts a structured, confidence-scored skill profile from a resume, parses a posting into required and preferred qualifications, and produces a 0-100 compatibility score that blends lexical similarity (TF-IDF), constraint-style skill coverage, and pre-trained Sentence-BERT semantic similarity. A Naive Bayes classifier trained on 2,484 real resumes across 24 job categories provides both a predicted job family and a per-skill confidence signal. The system is usable from a command-line demo and from an interactive web interface, and every number in this report is produced by a reproducible script in our repository (github.com/khangpt2k6/Intro_to_AI).

1. System Overview

The five-stage pipeline proposed at the start of the project is now fully implemented. The table below summarizes each stage and how it is built.

Pipeline stage	**Implementation**
Resume ingestion	Uploaded .pdf / .docx / .txt files are converted to clean, normalized text (pypdf, python-docx) so the rest of the pipeline sees the same shape of input regardless of source format.
Skill extraction + confidence	A 244-skill taxonomy (aliases matched with word-bounded regular expressions) covering all 24 dataset categories; each extracted skill gets a confidence in [0,1] blending mention frequency, experience context, and Naive Bayes category agreement (Section 3.2).
Job description parsing	Postings are split into hard constraints (required) and soft constraints (preferred), following the constraint-satisfaction framing from the proposal.
Compatibility scoring	A 0-100 score blending confidence-weighted skill coverage (0.4), corpus

	percentile of TF-IDF cosine similarity (0.3), and pre-trained Sentence-BERT semantic similarity (0.3).
Recommendation engine	Missing constraints ranked greedy best-first by the exact score gain each edit recovers; required skills weighted twice preferred ones.
Category classification	Multinomial Naive Bayes and Logistic Regression over TF-IDF features, evaluated on a stratified held-out test set (Section 4).

2. Dataset

We use the public Resume Dataset from Kaggle (snehaanbhawal/resume-dataset). It contains 2,484 real resumes provided as plain text and labeled into 24 job categories such as Information Technology, Finance, Engineering, Healthcare, Teacher, and Chef. The dataset is the training corpus for the TF-IDF vector space and the category classifier, and it supplies realistic test resumes for the matching demo.

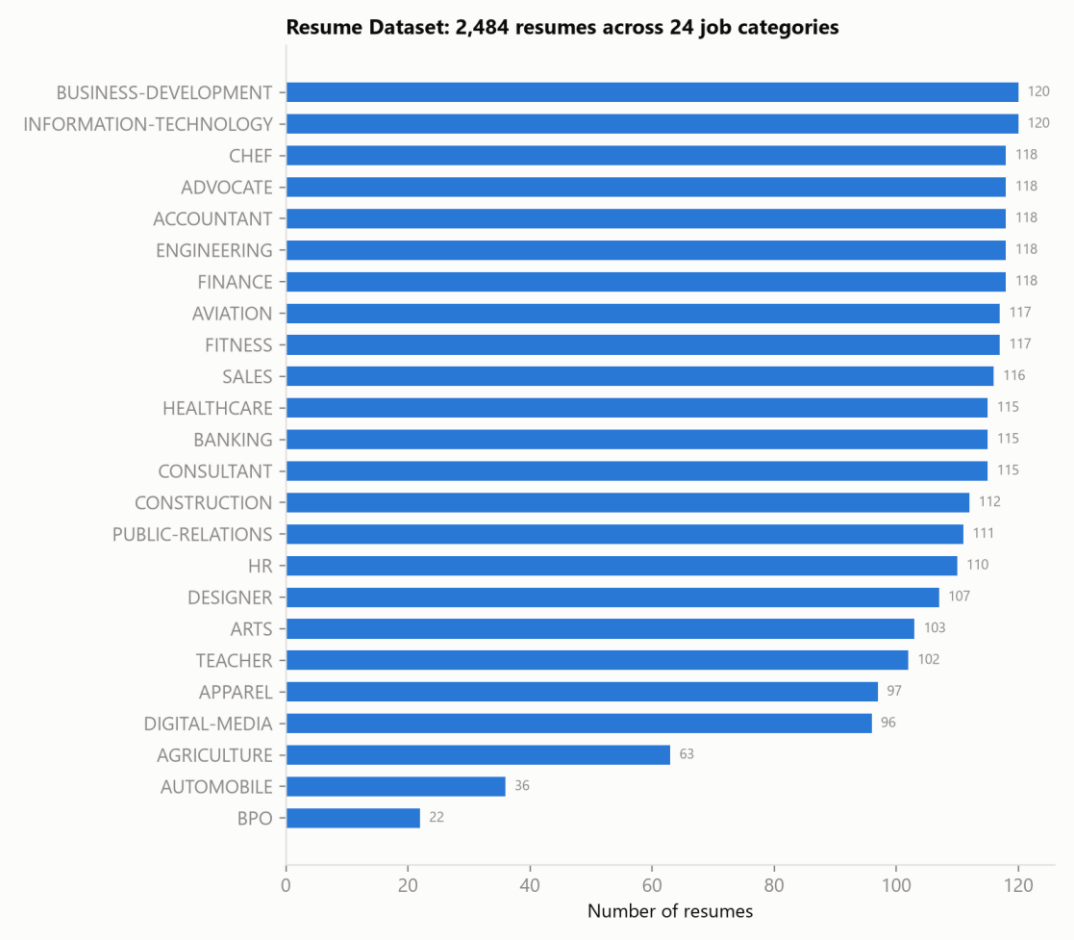

Figure 1. Distribution of the 2,484 resumes across the 24 job categories. The class imbalance (120 vs 22 resumes) is a challenge the classifier must handle.

3. Methods

3.1 Skill extraction and taxonomy

Skill mentions are normalized through a controlled vocabulary of 244 canonical skills, each with a list of surface-form aliases (for example 'JS' and 'Javascript' both map to JavaScript). Aliases are matched with case-insensitive, word-bounded regular expressions, and every alias is globally unique so a mention resolves to exactly one skill. The taxonomy was expanded from an initial IT/finance-focused set to span all 24 dataset categories (education, culinary, design, healthcare, legal, aviation, agriculture, and more); average skills extracted per resume rose several-fold for the previously under-covered categories, and every category now has at least one skill matched in 100% of its resumes.

3.2 Skill confidence (Naive Bayes agreement)

Binary alias matching tells us a skill is mentioned; it cannot tell us whether the resume genuinely demonstrates it. Each extracted skill therefore carries a confidence in [0,1] that

blends three signals: (1) mention frequency, so a skill named several times outweighs one named once; (2) experience context, which rewards mentions that sit next to action words ('built', 'developed', 'three years of ...') over mentions that only appear in a flat skills list; and (3) category agreement, a Naive Bayes signal. A Multinomial Naive Bayes classifier estimates P(category | resume); each skill has an expected category profile derived from the same model, and agreement is the cosine between the two distributions. A TensorFlow mention on a resume that reads as Information Technology is therefore trusted more than the same mention on an accountant resume. With the classifier the three signals are weighted 0.40 / 0.25 / 0.35; the matcher then credits each matched requirement by the resume's confidence in that skill, so a genuinely demonstrated skill contributes more to the coverage score than a name-dropped one.

3.3 Compatibility score

The final 0-100 score is a weighted sum of three components: confidence-weighted skill coverage (weight 0.4; required skills weighted 1.0, preferred 0.5), corpus percentile (weight 0.3; where the resume's TF-IDF cosine similarity to the posting ranks against all 2,484 corpus resumes), and semantic similarity (weight 0.3). Percentile calibration keeps the otherwise tiny raw cosine values (roughly 0.01 to 0.17) interpretable.

3.4 Sentence-BERT semantic similarity

TF-IDF only rewards exact lexical overlap, so a resume that says 'built REST services in Django' earns no credit against a posting asking for 'web backend development in Python'. To capture paraphrase, we add a semantic component: cosine similarity between sentence embeddings of the resume and the posting. We use a pre-trained Sentence-BERT model (all-MiniLM-L6-v2) strictly as a black-box embedder; consistent with the scope of the course, we do not fine-tune or otherwise train it. On the demo the semantic component tracks fit closely, ranging from 0.35 for the accountant posting to 0.68 for the software engineer posting.

3.5 Recommendation engine

Resume improvement is treated as a search problem: each missing constraint is a candidate edit scored by the exact coverage points it would recover, and edits are expanded best-first, largest gain first. Required skills dominate because they carry twice the weight of preferred ones.

4. Experimental Results

We trained two classifiers to predict a resume's job category from its text on an 80/20 stratified split (1,987 training, 497 test resumes). With 24 classes, random guessing would score about 4.2%. These classifier results are unchanged from the progress-report baseline, as expected: the classifier reads the raw resume text and is independent of the taxonomy and confidence work done since.

Model	Accuracy	Macro F1
Multinomial Naive Bayes	57.3%	0.494
Logistic Regression	68.6%	0.637

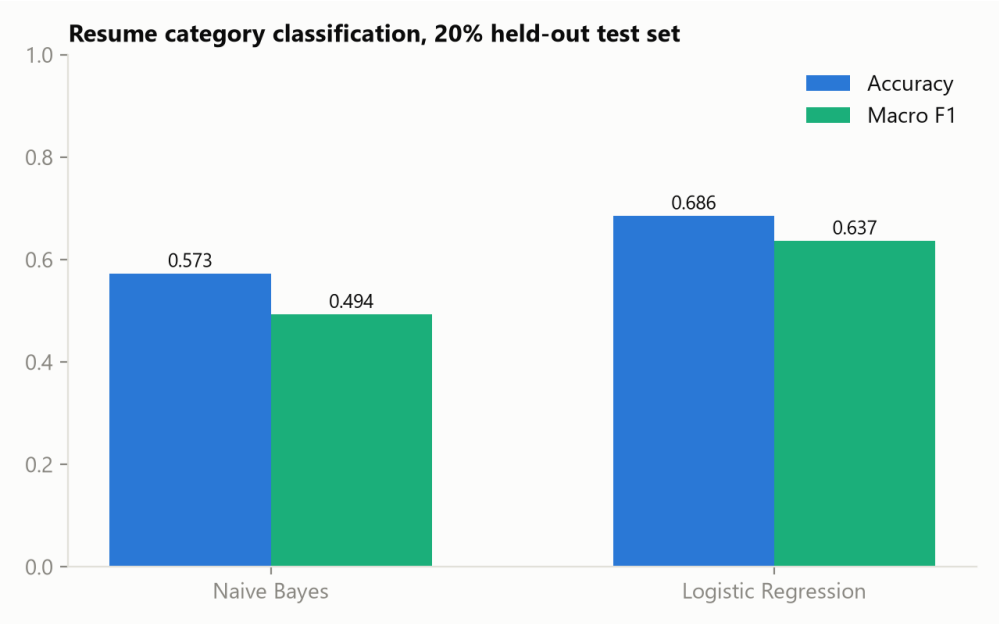

Figure 2. Both models beat the 4.2% random baseline by a wide margin; Logistic Regression is the stronger of the two.

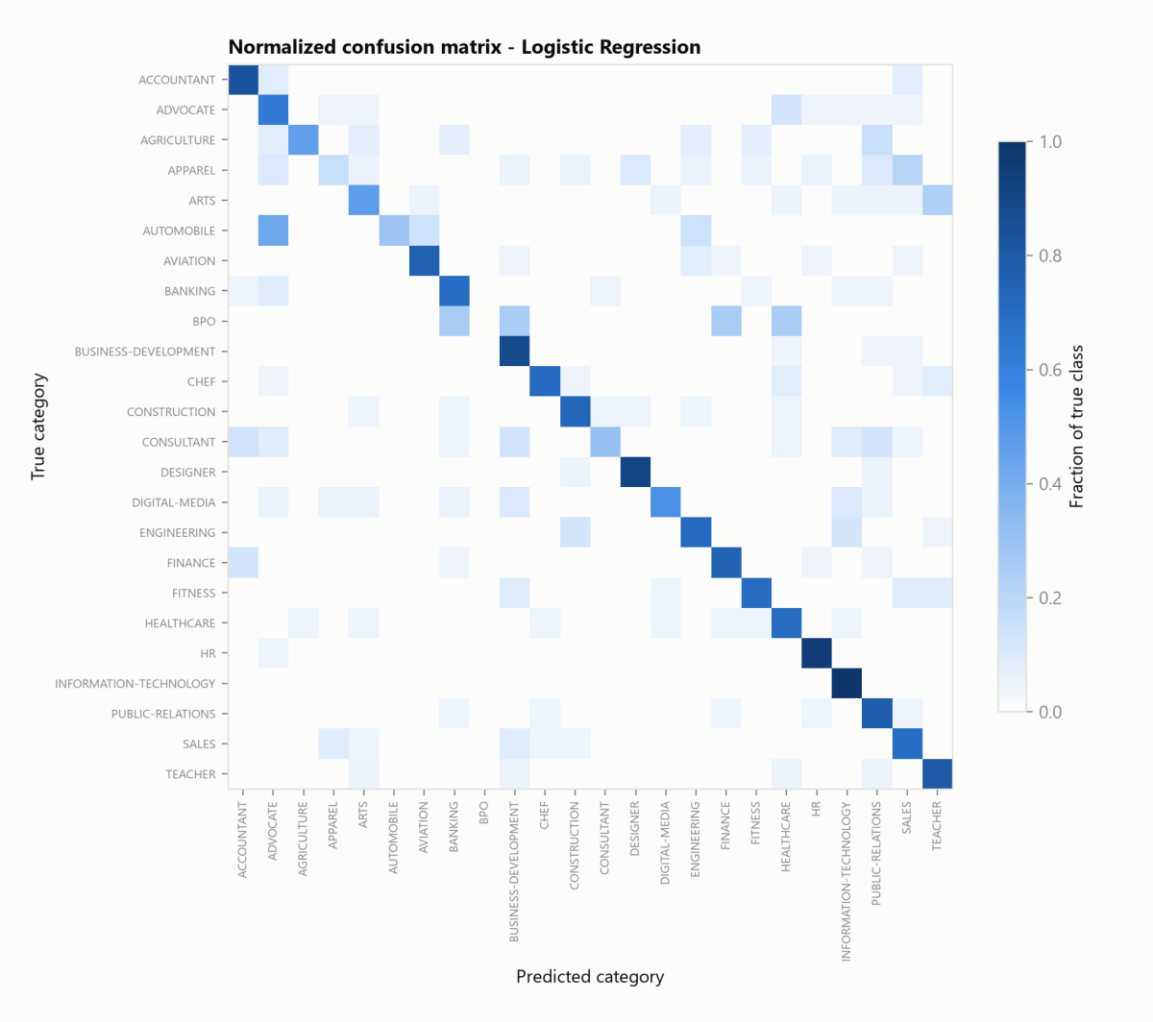

Figure 3. Normalized confusion matrix for Logistic Regression. The strong diagonal shows most categories are learned well; the remaining confusion concentrates in genuinely overlapping categories (Finance vs Accountant, Sales vs Business Development).

5. End-to-End Demo

To validate the whole pipeline we matched one real Information Technology resume from the dataset (ID 83816738) against three postings we wrote in realistic formats. The system behaves as a human reviewer would expect: the resume scores 71.6/100 against the software engineer posting, 47.4/100 against the data analyst posting, and only 13.5/100 against the accountant posting. Scores are lower than a naive count-based match would give because coverage is weighted by per-skill confidence.

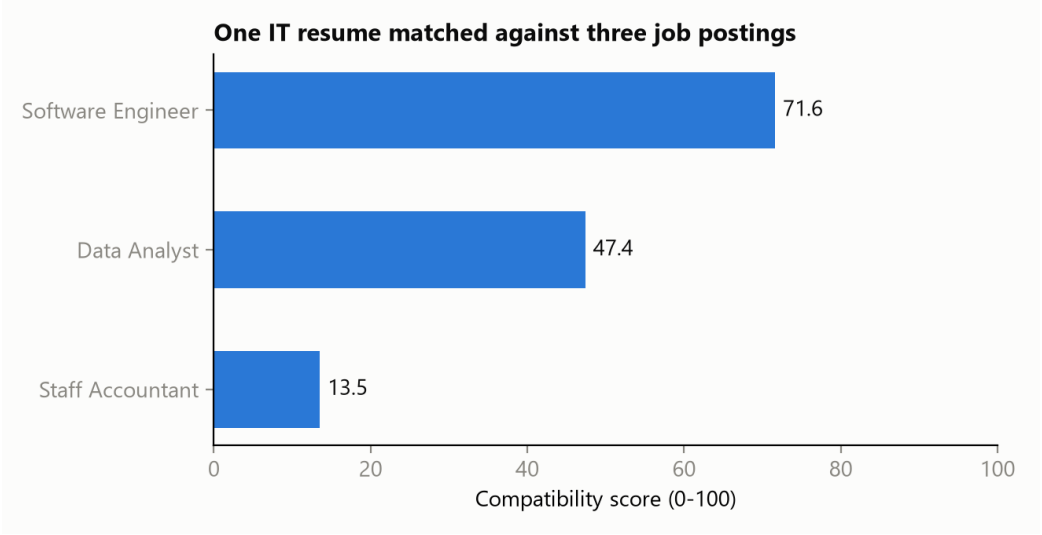

Figure 4. The same resume scored against three postings. The score cleanly separates the strong fit from the partial and poor fits.

For the software engineer posting the analyzer matched 6 of 8 required skills, each shown with its confidence: AWS (0.55), Agile (0.73), Git (0.80), Java (0.81), REST APIs (0.65), SQL (0.91). It flagged Docker, Python required skills as missing and ranked Docker, Python as the highest-value edits, ahead of preferred-skill edits like Kubernetes.

```
RESUME ANALYZER AND JOB MATCHER - DEMO
Test resume: dataset ID 83816738 (true category: INFORMATION-TECHNOLOGY)
======================================================================

Top extracted skills  (confidence = freq + context + NB agreement)
  SQL                  conf 0.91   x10, context 0.65, agreement 0.99
  PHP                  conf 0.88   x4, context 1.00, agreement 0.65
  Databases            conf 0.87   x6, context 0.65, agreement 0.88
  Spring               conf 0.86   x3, context 1.00, agreement 0.60
  Bootstrap            conf 0.82   x3, context 0.77, agreement 0.64
  Linux                conf 0.82   x2, context 0.65, agreement 0.97
  Java                 conf 0.81   x15, context 0.91, agreement 0.52
  HTML/CSS             conf 0.80   x7, context 0.80, agreement 0.57
  Git                  conf 0.80   x9, context 0.61, agreement 0.69
  Project Management   conf 0.78   x2, context 1.00, agreement 0.60
  CI/CD                conf 0.76   x4, context 0.82, agreement 0.45
  Microservices        conf 0.75   x3, context 0.77, agreement 0.46

JOB: Staff Accountant
  Compatibility score : 13.5/100
  Skill coverage      : 0%  (cosine 0.009, beats 10% of corpus)
  Semantic similarity : 0.348  (pre-trained Sentence-BERT embedding cosine)
  Required matched    : (none)
  Required MISSING    : Accounting, Excel, Financial Reporting, GAAP
  Preferred matched   : (none)
  Top recommended edits (estimated score gain):
    + Accounting           [required]  +5.7 pts
    + Excel                [required]  +5.7 pts
    + Financial Reporting  [required]  +5.7 pts
    + GAAP                 [required]  +5.7 pts
    + Auditing             [preferred]  +2.9 pts

JOB: Data Analyst
  Compatibility score : 47.4/100
  Skill coverage      : 14%  (cosine 0.045, beats 94% of corpus)
  Semantic similarity : 0.443  (pre-trained Sentence-BERT embedding cosine)
  Required matched    : SQL (0.91)
  Required MISSING    : Data Analysis, Excel, Pandas, Python
  Preferred matched   : Communication (0.50)
  Top recommended edits (estimated score gain):
    + Data Analysis        [required]  +5.0 pts
    + Excel                [required]  +5.0 pts
    + Pandas               [required]  +5.0 pts
    + Python               [required]  +5.0 pts
    + ETL                  [preferred]  +2.5 pts

JOB: Software Engineer - Backend
  Compatibility score : 71.6/100
  Skill coverage      : 53%  (cosine 0.174, beats 100% of corpus)
  Semantic similarity : 0.681  (pre-trained Sentence-BERT embedding cosine)
  Required matched    : AWS (0.55), Agile (0.73), Git (0.80), Java (0.81), REST APIs (0.65), SQL (0.91)
  Required MISSING    : Docker, Python
  Preferred matched   : CI/CD (0.76), Linux (0.82), React (0.63)
  Top recommended edits (estimated score gain):
    + Docker               [required]  +3.8 pts
    + Python               [required]  +3.8 pts
    + Kubernetes           [preferred]  +1.9 pts
    + Machine Learning     [preferred]  +1.9 pts
```

Figure 5. Console output for the demo resume across all three postings, including the extracted skills with confidence and the ranked edit recommendations.

6. Web Interface

The system is also usable from a Streamlit web app (streamlit run app.py). A user uploads a resume file or pastes resume text, picks or pastes a job description, and sees the compatibility score with its component breakdown, the matched and missing skills with

confidence, the Naive Bayes predicted job category, and the ranked recommended edits. The corpus, matcher, and models are cached so they load once per session.

Deploy

Resume

Upload a resume (.pdf, .docx, .txt)

Upload 200MB per file • PDF, DOCX, TXT

...or paste the resume text

```
Experience:
- Built and deployed Java and Spring Boot services to AWS, containerized with Docker.
- Developed SQL and PostgreSQL data models and designed relational databases.
- Used Git and GitHub for version control and set up CI/CD pipelines with Jenkins.
- Worked in an Agile Scrum team and led code reviews.
- Built internal tools with Python and Node.js and some React frontend work.
- Administered Linux servers; experience with REST APIs and web services.

Skills: Java, Python, SQL, REST APIs, Docker, AWS, Git, CI/CD, Linux, Agile, React, Spring
```

Job description

Prefill a sample posting

(none)

Job description

```
- 2+ years of professional experience with Python or Java
- Strong SQL skills and experience with relational databases
- Experience building and consuming REST APIs
- Proficiency with Git and code review workflows
- Experience with Docker and deploying services to AWS
- Comfortable working in an Agile team

Preferred Qualifications:
- Experience with Kubernetes in production
- Familiarity with React or another modern frontend framework
- Experience setting up CI/CD pipelines
- Exposure to machine learning systems
- Linux administration experience
```

Analyze match

Compatibility score	Skill coverage	Corpus percentile	Semantic (SBERT)
81.2/100	74%	100%	0.72

Score = 0.4 x coverage + 0.3 x percentile + 0.3 x SBERT. TF-IDF cosine to this job = 0.579.

Predicted resume category (Naive Bayes): ENGINEERING (50%), INFORMATION-TECHNOLOGY (33%), CONSULTANT (12%)

Matched skills

- **AWS** (required) - confidence 0.71
- **Agile** (required) - confidence 0.92
- **Docker** (required) - confidence 0.62
- **Git** (required) - confidence 0.89
- **Java** (required) - confidence 0.87
- **Python** (required) - confidence 0.87
- **REST APIs** (required) - confidence 0.93
- **SQL** (required) - confidence 0.80
- CI/CD (preferred) - confidence 0.92
- Linux (preferred) - confidence 0.74
- React (preferred) - confidence 0.71

Missing skills

- Kubernetes (preferred)
- Machine Learning (preferred)

Top recommended edits

skill	kind	estimated gain (pts)
Kubernetes	preferred	1.9000
Machine Learning	preferred	1.9000

Figure 6. The web interface, showing an IT resume matched against a backend software engineer posting: score, component breakdown, predicted category, matched/missing skills with confidence, and recommended edits.

7. Challenges

- **Class imbalance in the dataset.** The largest categories have 120 resumes while the smallest (BPO) has only 22. We report macro F1 (which weights every class equally) next to raw accuracy, and use a stratified split.
- **Semantically overlapping categories.** Classification error concentrates in category pairs that share vocabulary (Finance vs Accountant, Sales vs Business Development). This motivated adding Logistic Regression (+11 points over Naive Bayes) and the Sentence-BERT semantic signal in the matcher.
- **Distinguishing genuine skills from name-dropping.** A skill listed once in a skills block is weaker evidence than one used in described work. The confidence blend (frequency, experience context, and Naive Bayes category agreement) addresses this directly and feeds a confidence-weighted coverage score.
- **Short skill aliases cause false positives.** Aliases like 'R' or 'Go' match ordinary words, and generic terms cause cross-domain hits (an early version matched 'reporting' in 'financial reporting' as Journalism). We use word-bounded matching, prefer longer alias forms, and keep aliases globally unique, verified by automated tests.
- **Reproducibility and safety of the pipeline.** Continuous integration runs the test suite on every change, and the heavy Sentence-BERT import is deferred so the core package and tests run without the deep-learning stack.

8. Team Contributions

We split the work evenly (50/50), pairing on design decisions and dividing implementation by pipeline stage:

Martin Dang (50%)	**Tuan Khang Phan (50%)**
Skill taxonomy design and expansion to 24 categories; skill extraction and confidence blend; job constraint parser; recommendation engine; Sentence-BERT semantic similarity component	Dataset acquisition and preprocessing; TF-IDF matching engine and percentile calibration; Naive Bayes category and confidence model; classifier experiments; resume ingestion; Streamlit web interface
Joint: sample job postings, end-to-end testing, this report	Joint: system design, error analysis, this report

9. Conclusion

The completed system meets the goals set out in the proposal. It turns a free-text resume and job posting into an interpretable compatibility score and a ranked list of concrete improvements, combining classical NLP (TF-IDF, Naive Bayes) with a pre-trained transformer embedding used as a black box. The confidence-weighted coverage and the semantic signal make the score more faithful than lexical matching alone, and the web interface makes the tool usable by a non-technical user. Natural next steps are measuring extraction precision and recall on a hand-labeled sample and calibrating the score against human fit judgments.